

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

2. UTP Cable Validation

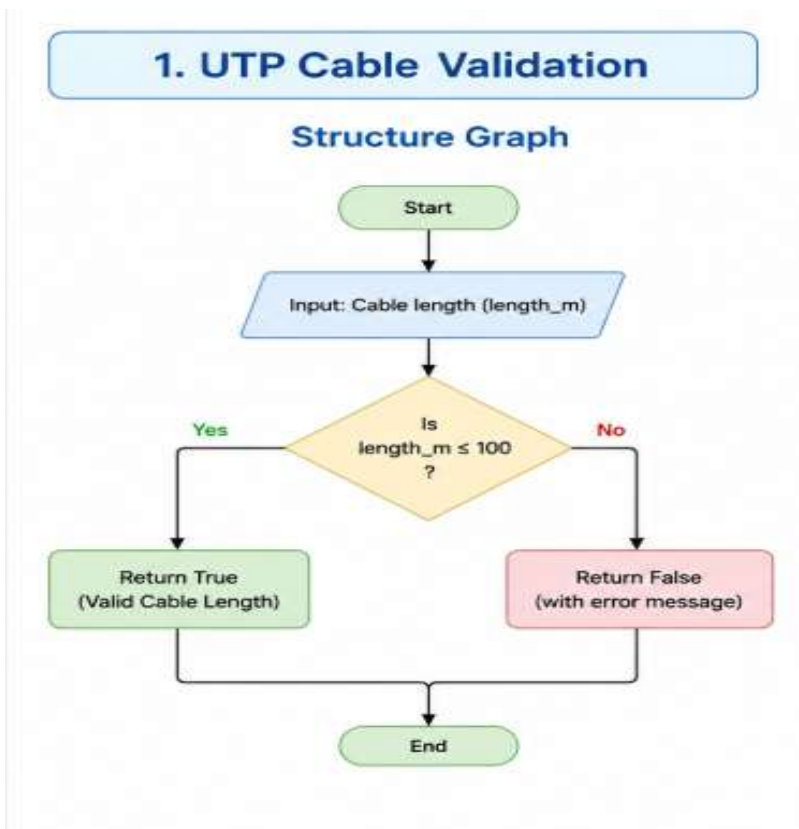
Aim

To validate whether a user-entered UTP cable segment meets the maximum Ethernet length limit of 100 meters.

Procedure

1. Define the `validate_segment(length_m)` function.
2. Accept cable length values from the user as input.
3. Check whether each length is within 100 meters.
4. Return True for valid lengths and an error message for invalid lengths.
5. Test the function with at least three input values.

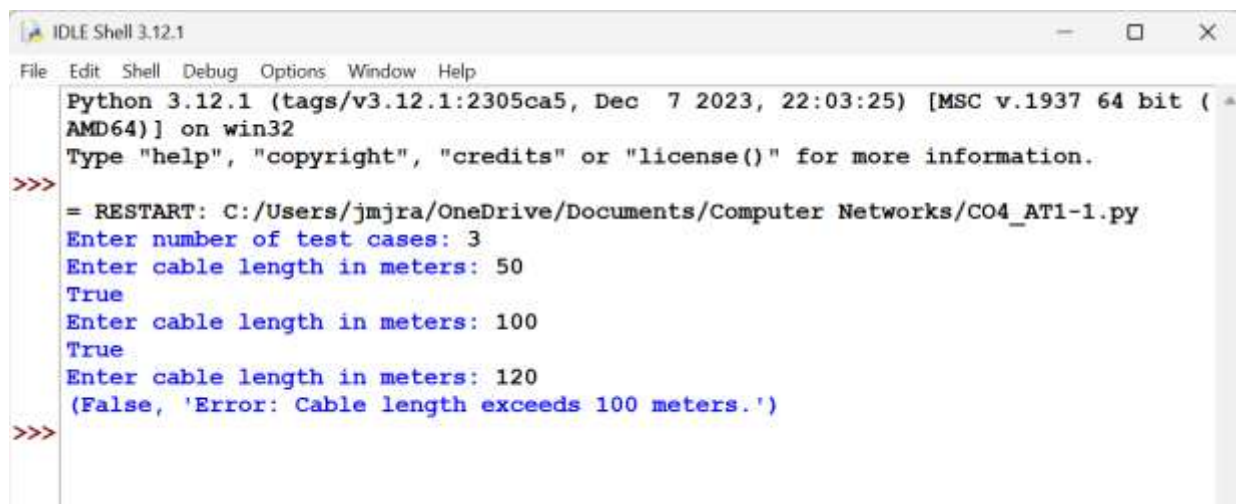
Graph



Code

```
def validate_segment(length_m):  
    if length_m <= 100:  
        return True  
    return False, "Error: Cable length exceeds 100 meters."  
  
n = int(input("Enter number of test cases: "))  
  
for i in range(n):  
    length = float(input("Enter cable length in meters: "))  
    print(validate_segment(length))
```

Output



```
IDLE Shell 3.12.1  
File Edit Shell Debug Options Window Help  
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>> = RESTART: C:/Users/jmjra/OneDrive/Documents/Computer Networks/CO4_AT1-1.py  
Enter number of test cases: 3  
Enter cable length in meters: 50  
True  
Enter cable length in meters: 100  
True  
Enter cable length in meters: 120  
(False, 'Error: Cable length exceeds 100 meters.')
```

Result

The program successfully validates user-entered cable lengths against the 100-meter Ethernet limit.

3. CSMA/CD Simulation

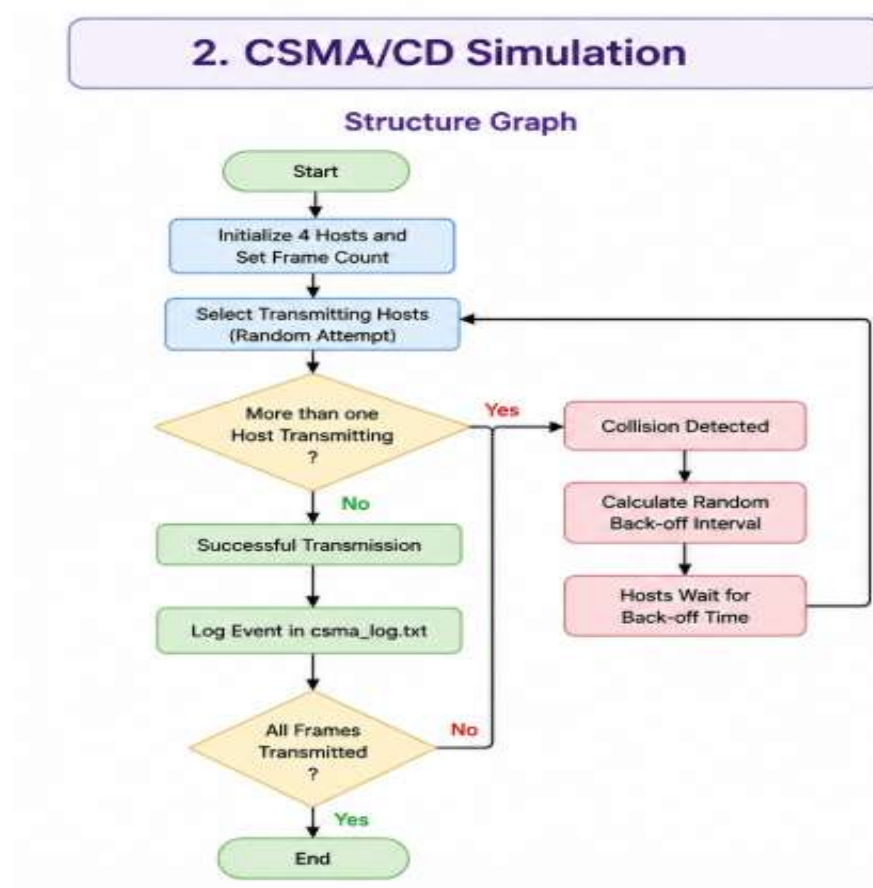
Aim

To simulate CSMA/CD frame transmission in a four-host star topology with collision detection, random back-off, and event logging.

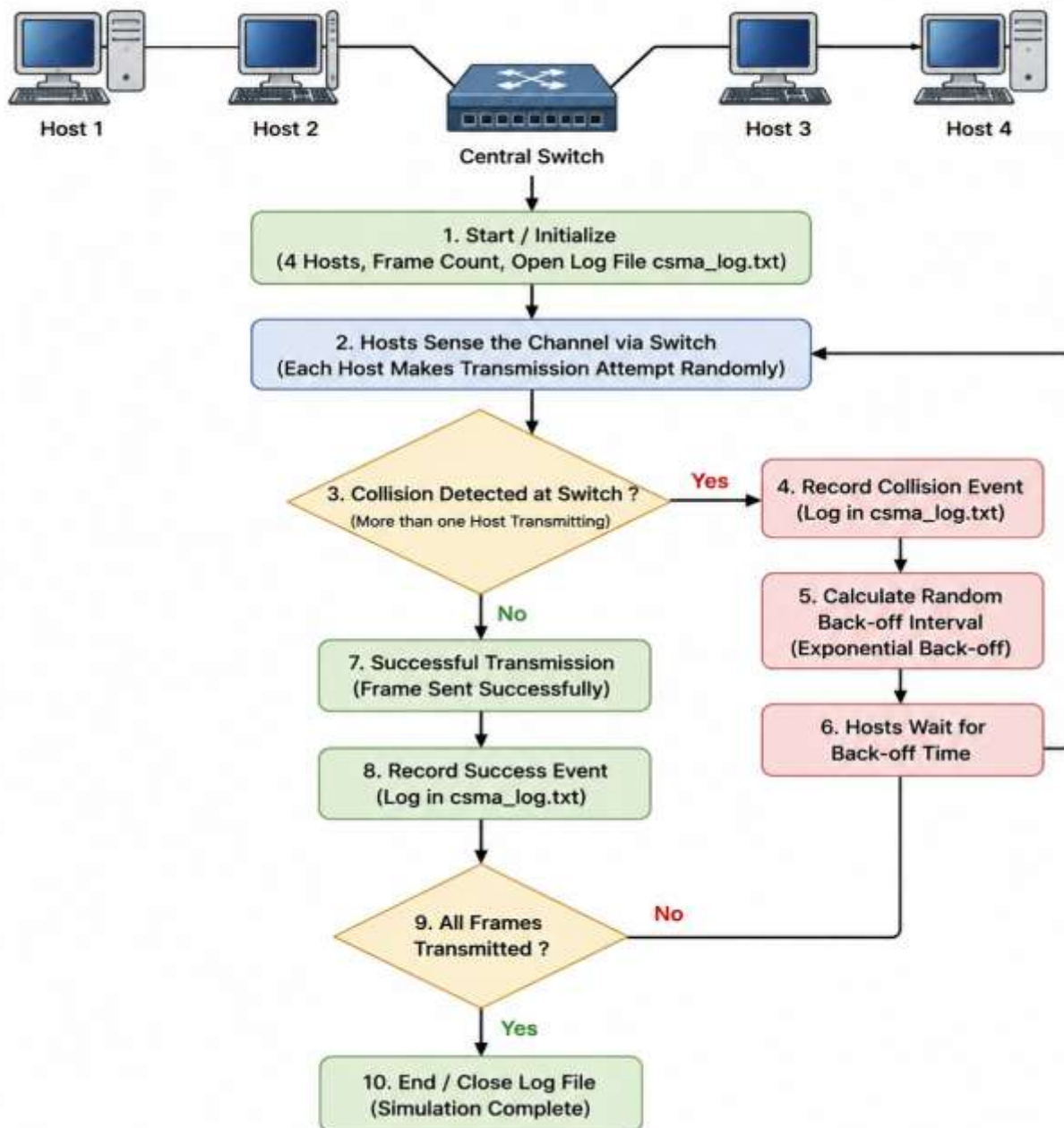
Procedure

1. Create four hosts connected through a central switch.
2. Generate random frame transmission attempts by the hosts.
3. Detect collisions when multiple hosts transmit simultaneously.
4. Calculate a random back-off interval after each collision.
5. Record attempts, collisions, and back-off intervals in csma_log.txt.

Graph



CSMA/CD Simulation – Topology Flow Chart



Code

```
import random
```

```
import time
```

```
hosts = ["Host1", "Host2", "Host3", "Host4"]
```

```
frames = 3
```

```
log_file = "csma_log.txt"
```

```

with open(log_file, "w") as log:
    log.write("CSMA/CD Simulation Log\n")
    log.write("=====\n")

for frame in range(1, frames + 1):
    pending = hosts.copy()
    attempts = {host: 0 for host in hosts}

    while pending:
        transmissions = {}

        for host in pending:
            if random.random() < 0.6:
                transmissions.setdefault(0, []).append(host)

        if not transmissions:
            continue

        active_hosts = transmissions[0]

        for host in active_hosts:
            attempts[host] += 1
            log.write(
                f'Frame {frame}: {host} transmission attempt "
                f'{attempts[host]}\n'
            )

        if len(active_hosts) == 1:
            host = active_hosts[0]
            log.write(f'Frame {frame}: {host} transmitted successfully\n")
            pending.remove(host)

        else:
            log.write(
                f'Frame {frame}: Collision detected among "
                f'{", ".join(active_hosts)}\n'
            )

```

)

for host in active_hosts:

collision_count = attempts[host]

backoff = random.randint(1, min(2 ** collision_count, 16))

log.write(

f'Frame {frame}: {host} back-off interval = "

f'{backoff} time units\n"

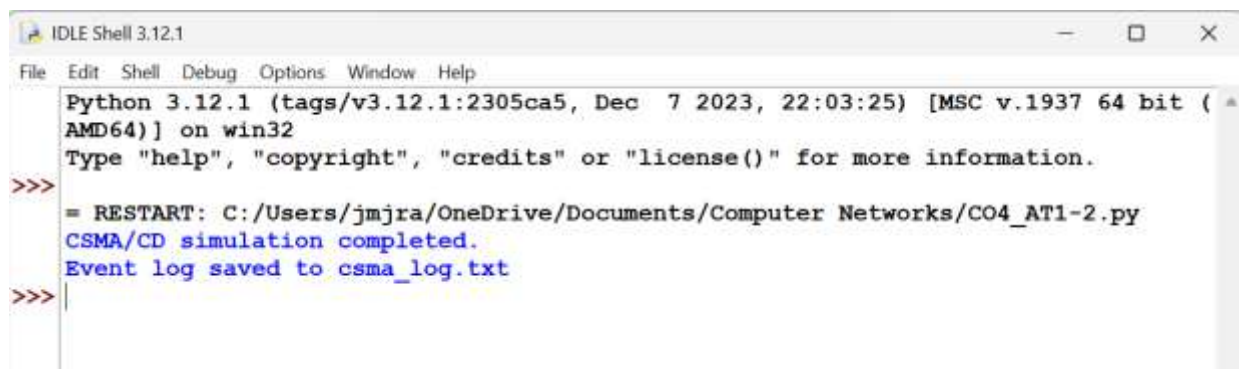
)

time.sleep(0.1)

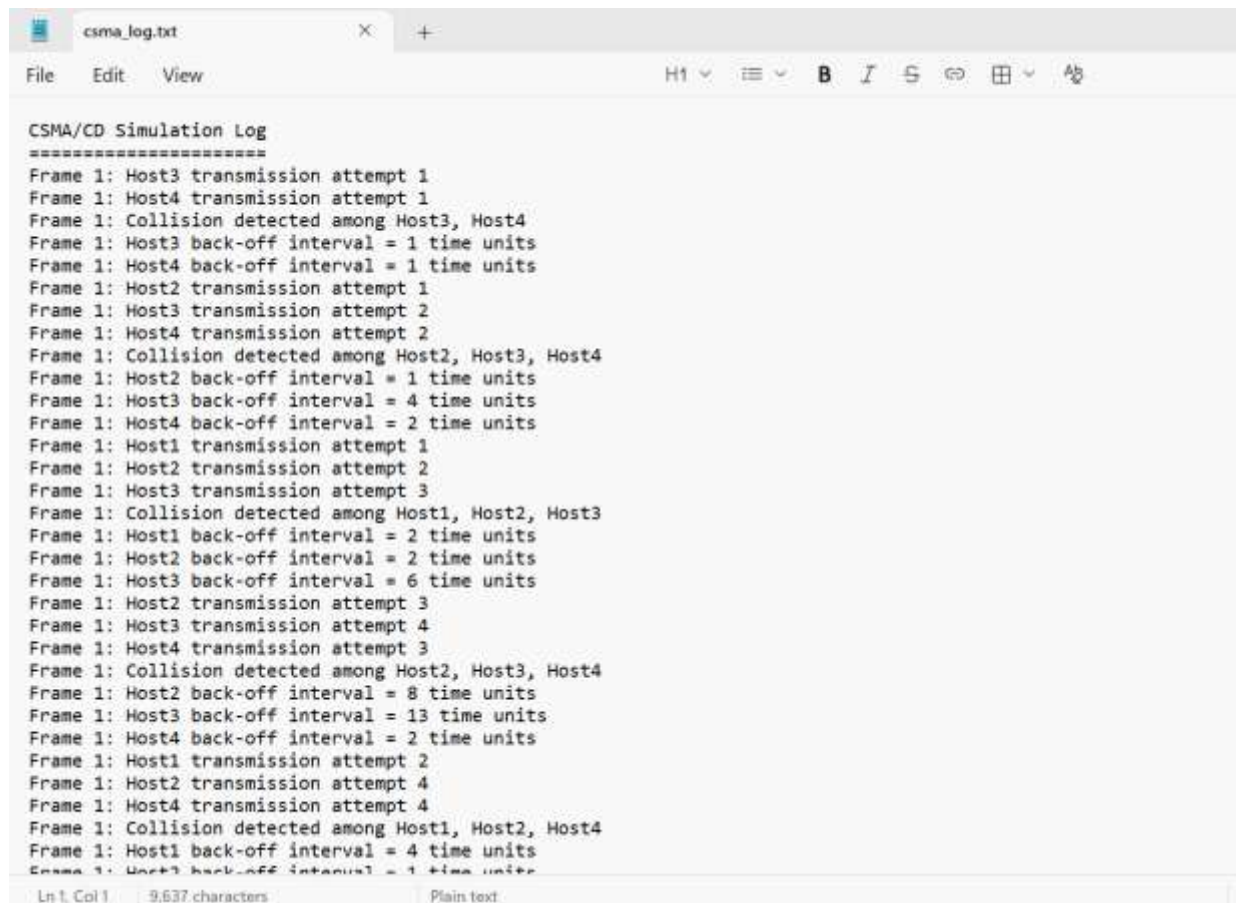
print("CSMA/CD simulation completed.")

print("Event log saved to csma_log.txt")

Output



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/jmjra/OneDrive/Documents/Computer Networks/CO4_AT1-2.py
CSMA/CD simulation completed.
Event log saved to csma_log.txt
>>>
```

```
csma_log.txt
File Edit View H1 B I S [ ] [ ]
CSMA/CD Simulation Log
=====
Frame 1: Host3 transmission attempt 1
Frame 1: Host4 transmission attempt 1
Frame 1: Collision detected among Host3, Host4
Frame 1: Host3 back-off interval = 1 time units
Frame 1: Host4 back-off interval = 1 time units
Frame 1: Host2 transmission attempt 1
Frame 1: Host3 transmission attempt 2
Frame 1: Host4 transmission attempt 2
Frame 1: Collision detected among Host2, Host3, Host4
Frame 1: Host2 back-off interval = 1 time units
Frame 1: Host3 back-off interval = 4 time units
Frame 1: Host4 back-off interval = 2 time units
Frame 1: Host1 transmission attempt 1
Frame 1: Host2 transmission attempt 2
Frame 1: Host3 transmission attempt 3
Frame 1: Collision detected among Host1, Host2, Host3
Frame 1: Host1 back-off interval = 2 time units
Frame 1: Host2 back-off interval = 2 time units
Frame 1: Host3 back-off interval = 6 time units
Frame 1: Host2 transmission attempt 3
Frame 1: Host3 transmission attempt 4
Frame 1: Host4 transmission attempt 3
Frame 1: Collision detected among Host2, Host3, Host4
Frame 1: Host2 back-off interval = 8 time units
Frame 1: Host3 back-off interval = 13 time units
Frame 1: Host4 back-off interval = 2 time units
Frame 1: Host1 transmission attempt 2
Frame 1: Host2 transmission attempt 4
Frame 1: Host4 transmission attempt 4
Frame 1: Collision detected among Host1, Host2, Host4
Frame 1: Host1 back-off interval = 4 time units
Frame 1: Host2 back-off interval = 1 time units
Ln 1, Col 1 9,637 characters Plain text
```

Result

The program successfully simulates CSMA/CD by detecting collisions, applying random back-off intervals, retrying transmissions, and recording all events in csma_log.txt. The back-off mechanism reduces repeated collisions and helps coordinate successful Ethernet frame transmission.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done